

BUILDING LLM-POWERED APPLICATIONS · LECTURE 3

Backend & REST API

Put the model behind your own server: validate, rate-limit, and hide the key.

By the end you can: hide the key · pick verbs/status codes · model requests/responses with Pydantic · ship
`POST /api/chat` with errors & rate limits

Press `F` for fullscreen · navigate with `←` `→`

Your React app calls the OpenAI API directly from the browser with your API key in the JS bundle. What's the worst realistic consequence?

Nothing — the key is only in compiled code, users can't read it

Anyone can open dev-tools, copy the key, and spend your money ✓

The app will be slower because the browser adds latency

CORS will block the request so it simply won't work

Why put the LLM behind your backend?

So far we called the model straight from a script. A real app never ships the API key to the browser. The server sits **in the middle**:

It protects

- Hide the key — it never reaches the client
- Rate-limit per user, cap cost
- Validate input before spending tokens

It controls

- Log requests for debugging & eval
- Inject system prompts & context
- Swap models without touching the UI

The browser talks to *your* API; only your API talks to the model.

You hide the API key on a server. Which of these does that ALONE still not give you?

A place to validate input before spending tokens

Per-user rate limiting and request logging

A faster model — the backend speeds up generation itself ✓

Freedom to swap models without shipping new frontend code

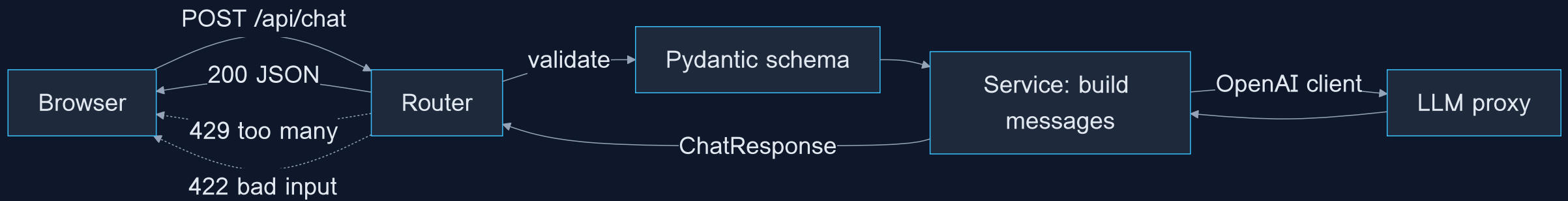
Think · pair · share: trace the request

Put these five stages in the order a chat request visits them, and label each arrow with what can go wrong there:

Browser · Router · Pydantic · Service · LLM proxy

Where does a 422 come from? A 429? A 502? Decide with your neighbour — then draw it on the next slide.

Request Flow



A backend is a process that answers requests

Your scripts so far *ran and exited*. A server is different: a long-running process that **waits**, and for every incoming HTTP request sends back one HTTP response.

Request (what the browser sends)

```
POST /api/chat HTTP/1.1
Host: courses.nat-d.uk
Content-Type: application/json

{"message": "Hi", "temperature": 0.7}
```

Response (what your API sends back)

```
HTTP/1.1 200 OK
Content-Type: application/json

{"reply": "Hello!", "model": "gemma-4-E4B-it"}
```

A **verb** + **path** + optional JSON **body** go in; a **status line** + JSON come back. Everything else this lecture is shaping those two boxes.

Wire Bytes

```
# No FastAPI, no network: an HTTP message is just text. Here we BUILD the exact  
# bytes the browser sends to POST /api/chat, then PARSE a response the way a client  
# does. This is what FastAPI serializes for you. Ctrl/Cmd+Enter to run.
```

```
import json
```

```
# --- build the request the browser would send ---
```

```
body = json.dumps({"message": "Hi", "temperature": 0.7})
```

```
request = (
```

```
    "POST /api/chat HTTP/1.1\r\n"
```

```
    "Host: courses.nat-d.uk\r\n"
```

```
    "Content-Type: application/json\r\n"
```

```
    f"Content-Length: {len(body)}\r\n"
```

```
    "\r\n"
```

```
    f"{body}"
```

```
)
```

```
print("--- REQUEST ---")
```

```
print(request)
```

```
# --- parse a raw response the way a client does: status line, then JSON body ---
```

Why the browser sometimes blocks your API

A browser only lets a page call an API on a **different origin** (scheme + host + port) if that API explicitly allows it. That's **CORS**.

Different origin → blocked

```
page: https://app.example.com
api : https://api.example.com
# browser sends a preflight OPTIONS;
# no Access-Control-Allow-Origin -> blocked
```

Same origin → nothing to do

```
page: https://courses.nat-d.uk/
api : https://courses.nat-d.uk/api
# this app serves the SPA + API from
# ONE origin -> no CORS to configure
```

FastAPI serves the built frontend (Vite proxies `/api` in dev), so we dodge CORS entirely. You'd add it only if the frontend were hosted separately.

REST in 90 seconds

A REST API exposes **resources** at URLs; the HTTP **verb** says what to do with one.

Verbs

- `GET` — read, no side effects
- `POST` — create / run an action
- `PUT` / `PATCH` — update
- `DELETE` — remove

Status codes

- **2xx** ok — `200`, `201`
- **4xx** your fault — `400`, `401`, `422`, `429`
- **5xx** our fault — `500`, `502`

A chat turn *does work* (spends tokens), so it's a `POST`, not a `GET`.

Before we name the rules: a chat turn spends tokens and runs the model. Which HTTP verb fits POST /api/chat best?

GET — it just fetches a reply, like reading a page

POST — it performs an action with side effects (spends tokens) ✓

PUT — it replaces a resource at a known URL

DELETE — it removes the previous conversation

Picking the honest 5xx

When something past the client breaks, the status code names **who failed and how** — so the frontend can branch on it.

Your code's fault

- **500** — an unhandled bug in *your* handler or service

An upstream's fault

- **502** — the model proxy answered with an error
- **504** — the model *timed out*; nothing came back in time

Why distinguish?

A **504** is worth a quiet retry — the request may just have been slow. A **500** means *you* shipped a bug; retrying won't help. Same red color to a user, very different action.

Catch the upstream timeout as **504**; let everything else fall to a logged **502** / **500**.

POST /api/chat with body {"message":123,"temperature":0.7} — valid JSON, but message is a number. What do FastAPI + Pydantic return?

400 Bad Request — the JSON is wrong

422 Unprocessable Entity — JSON parsed, but a field failed validation ✓

200 OK — FastAPI coerces 123 to "123"

500 Internal Server Error — the handler crashes on the bad type

Model the request & response

Pydantic schemas are your **contract**. FastAPI parses the JSON body into them and rejects anything that doesn't fit — before your code runs.

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    message: str = Field(min_length=1, max_length=2000)
    temperature: float = Field(default=0.7, ge=0.0, le=2.0)

class ChatResponse(BaseModel):
    reply: str
    model: str
```

A 3000-char message or a missing field never reaches the model — it's a **422** with a clear error.

Predict: you give the handler a ChatRequest schema. A request arrives with message missing entirely. When does it get rejected?

Inside your handler code, after it starts running

At the edge — FastAPI/Pydantic rejects it before your handler runs ✓

Never — Pydantic fills missing fields with a default empty string

Only the LLM proxy notices, and it returns the error

Pydantic Validation

```
# pydantic IS installed in the sandbox. This is the SAME ChatRequest the  
# FastAPI handler uses. Press Ctrl/Cmd+Enter to run.  
from pydantic import BaseModel, Field, ValidationError  
  
class ChatRequest(BaseModel):  
    message: str = Field(min_length=1)  
    temperature: float = Field(0.7, ge=0, le=2)  
  
# A GOOD request parses cleanly into a typed object.  
good = ChatRequest.model_validate({"message": "Hello!", "temperature": 0.7})  
print("OK ->", good)  
  
# A BAD request raises ValidationError. FastAPI turns exactly this  
# ValidationError into the 422 response body the client receives.  
try:
```

The `/api/chat` handler

The router stays thin: declare verb + path, take the typed body, return a typed model.

```
from fastapi import APIRouter, HTTPException

router = APIRouter(prefix="/api")

@router.post("/chat", response_model=ChatResponse)
async def chat(req: ChatRequest, user = Depends(current_user)):
    if not await rate_limiter.allow(user.id):
        raise HTTPException(429, "slow down")
    reply = await chat_service.complete(req)
    return ChatResponse(reply=reply, model=MODEL)
```

`response_model` validates & documents the output. Business logic lives in the [service](#), not here.

Why the handler is `async`

One event loop serves **every** request. A blocking call there — like the `sync` `OpenAI` client waiting seconds for the model — freezes **all** other requests until it returns.

Blocks the loop

```
# sync client inside async handler
reply = client.chat.completions.create(...)
# loop stuck here for seconds;
# nobody else gets served
```

Frees the loop

```
# async client, awaited
reply = await aclient.chat \
    .completions.create(...)
# loop serves others while
# the model thinks
```

Use `AsyncOpenAI` and `await` it — or wrap a sync call in `asyncio.to_thread(...)`. That's what "async everywhere" buys you.

House style: router → service → repository

The reference app's `examples/` wires all three layers together — a real example you can read.

router

HTTP only: path, verb, status codes.
Maps errors to responses. No business logic.

service

Business logic: build messages, call the LLM, rate-limit, log.

repository

Pure data access (the DB). The chat endpoint may not need one yet — but conversation history would live here.

Thin edges, fat middle: easy to test the logic without HTTP.

The repository layer (ORM)

Persist conversation history? That read/write lives in the **repository** — pure data access, no business logic. Built on an **ORM**: typed rows, composable queries.

```
class Conversation(Base):
    __tablename__ = "conversations"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[str]
    message: Mapped[str]
    reply: Mapped[str]
```

```
class ConversationRepo:
    # NO business logic
    def add(self, turn): ...
    def recent(self, uid):
        return select(
            Conversation)...
```

The table it maps is what the Alembic migration creates. Runnable: reference app's `examples/orm_repository.py` & `examples/layering.py`.

A Pydantic 422 can flag several bad fields at once, while your own `HTTPException(502, "upstream error")` carries one message. So how do their error bodies differ in shape?

Both are always a single plain message — the frontend treats them identically

The 422 is a list of per-field errors; the `HTTPException` is one plain message ✓

The 422 is one message; the `HTTPException` is a list of fields

They use different key names, so the frontend can't read either one

In the router → service → repository style, which layer should reject a 3000-character message?

The repository — it's closest to the stored data

The router/edge — Pydantic validates the request body before the handler runs ✓

The service — business logic decides what's valid

The LLM — it'll just refuse or truncate a too-long prompt

Predict before you run

On the next slide you'll run `chat_endpoint(...)`. First commit to a prediction:

```
chat_endpoint({"message":  
"..."})
```

status? reaches the model?

```
chat_endpoint({"message":  
"    "})
```

status? does the empty one reach the model?

Decide, then run.

Handler Sim

```
# We can't import fastapi in the sandbox, so we SIMULATE the handler: a plain  
# function that takes a request dict, validates it, calls the chat API, and  
# returns a response dict (just like /api/chat would). Press Ctrl/Cmd+Enter.
```

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
MODEL = "gemma-4-E4B-it"
```

```
def chat_endpoint(request: dict) -> dict:
```

```
# 1. Validate the request (FastAPI/Pydantic does this for you in prod).
```

```
msg = request.get("message")
```

```
if not isinstance(msg, str) or not msg.strip():
```

```
    return {"status": 422, "error": "message is required"}
```

```
if len(msg) > 2000:
```

```
    return {"status": 422, "error": "message too long"}
```

Think · pair · share: design the contract

Your `POST /api/chat` must support a system prompt the server injects and (later) conversation history. With your neighbour, decide the request body:

`message` · `temperature` · `conversation_id?` · `system_prompt?`

Which fields does the **client** get to set, and which must the **server** own (and why)? Should the system prompt come from the client at all? What new status code appears once `conversation_id` can be unknown?

Testing code that calls a model

The model is **non-deterministic** — ask twice, get different words. So you can't assert the reply *equals* a fixed string.

Split the problem

The scaffolding is deterministic — messages, validation, 422/502, the rate limiter. Unit-test it with the model mocked.

The **answer quality** is not — judge it with an eval (Lecture 8), never a unit test.

Assert properties, not equality

```
# ✗ brittle — the wording varies
assert reply == "The total is 96 baht."

# ✓ a property of the output
assert "96" in reply
```

Mock the model at the **boundary**: inject a fake client that returns a canned reply — fast, offline, repeatable.

Stub And Assert

```
# Unit-test LLM code with NO pytest and NO network: stub the model, then
# assert PROPERTIES of the deterministic scaffolding around it.

def build_messages(system, user):          # the part worth testing
    return [{"role": "system", "content": system},
            {"role": "user", "content": user}]

def fake_model(messages):                 # stub: stands in for the real client
    return "The total is 96 baht."

def handle(system, user):
    msgs = build_messages(system, user)
    reply = fake_model(msgs)              # prod: client.chat.completions.create(...)
    return {"messages": msgs, "reply": reply}

out = handle("You are terse.", "Sum 90 and 6 in baht.")
assert [m["role"] for m in out["messages"]] == ["system", "user"]    # right shape
```

You're testing `POST /api/chat`. Which check belongs in a **UNIT/e2e** test (model mocked) — not an eval?

The endpoint returns 422 when `message` is empty ✓

The answer is factually correct and well-written

The answer never invents a source it wasn't given

The reply sounds polite and on-brand

Errors are part of the API

Map every failure to a **status code the client can act on** — and never leak the API key or a stack trace.

```
try:
    reply = await call_model(req)
except httpx.TimeoutException:
    raise HTTPException(504, "model timed out")
except Exception:
    log.exception("chat failed")          # full detail in YOUR logs
    raise HTTPException(502, "upstream error") # vague to client
```

The response **body** always has a **detail** field:

```
# 422 from Pydantic — a list of per-field errors
{"detail": [{"loc": [...], "msg": ..., "type": ...}]}
# your HTTPException(502, ...) — a plain string
{"detail": "upstream error"}
```

On a **429** also send a **Retry-After** header so the client knows how long to back off. Log details server-side; stay vague to the user.

What happens if the LLM proxy itself returns an error while your handler is calling it? Which status should your /api/chat send back?

200 OK — hide the failure and return an empty reply

422 Unprocessable Entity — the client sent something wrong

502 Bad Gateway — an upstream you depend on failed, not the client ✓

404 Not Found — the model endpoint could not be located

The `detail` field is a contract

Every error FastAPI returns carries a `detail` — but its `type` tells the frontend what to render. Next lecture's UI branches on exactly this.

422 → a LIST

```
{"detail": [  
  {"loc": ["body", "message"],  
   "msg": "too short",  
   "type": "string_too_short"}  
]}
```

→ show a per-field form error.

HTTPException → a STRING

```
{"detail": "upstream error"}
```

→ show a single toast.

`isinstance(detail, list)` vs `str` is the whole branch. Keep the shape stable and the frontend stays simple.

A token-bucket rate limit

Each user gets a bucket of **tokens** that refills over time. Every request spends one; an empty bucket means **429**.

```
# 30 requests / minute -> refill 0.5 tokens/sec, cap 30
def allow(bucket, rate=0.5, cap=30):
    now = time.monotonic()
    bucket.tokens = min(cap, bucket.tokens + (now - bucket.last) * rate)
    bucket.last = now
    if bucket.tokens >= 1:
        bucket.tokens -= 1
        return True
    return False    # -> 429 Too Many Requests
```

This is exactly how the reference app limits clients (`_rate_ok` in `app/main.py`; in-process — swap to Redis for multiple replicas).

Token Bucket Run

```
# The token bucket from the lecture, RUNNABLE. No FastAPI needed: a dict of  
# buckets keyed by client id is exactly what the reference app keeps in-process  
# (_rate_ok in app/main.py).  
# 30 req/min -> rate=0.5 tokens/sec, cap=30. Press Ctrl/Cmd+Enter.  
import time  
from dataclasses import dataclass  
  
@dataclass  
class Bucket:  
    tokens: float  
    last: float  
  
buckets: dict[str, Bucket] = {}  
  
def check(user_id: str, rate: float = 0.5, cap: float = 30) -> dict:  
    b = buckets.setdefault(user_id, Bucket(cap, time.monotonic()))
```

Retrying a `POST` can double-charge you

A `GET` is safe to retry — no side effects. But each `POST /api/chat` spends tokens. If the reply is lost on the network, a blind retry runs the model *twice*.

The fix: an idempotency key

- Client sends a unique id with the request
- Server stores the result under that id
- A retry with the same key returns the stored result — no second model call

```
POST /api/chat
Idempotency-Key: 7f3c-...-a91
{"message": "Hi"}

# retry w/ same key ->
# cached reply, $0 extra
```

Stripe's API made this idea standard — see the further reading.

Schema changes: migrations, not hand-edits

When the DB schema must change, don't hand-edit prod tables. A [migration](#) is a versioned, **ordered**, **reversible** script — Alembic chains and replays them everywhere.

```
# three commands
alembic revision -m "add table"
alembic upgrade head
alembic downgrade -1
```

```
def upgrade():
    op.create_table(...)

def downgrade():
    op.drop_table(...)
```

The reference app's `alembic/` runs `alembic upgrade head` — its pgvector schema and code ship in lockstep.

A client POSTs to /api/chat with an empty message. Which status should the API return?

200 OK

404 Not Found

422 Unprocessable Entity ✓

500 Internal Server Error

A backend that holds up

- Keep secrets server-side. The browser never sees the API key.
- Validate at the edge. Typed Pydantic schemas reject bad input early.
- Pick honest status codes. 422 for bad input, 429 for rate limits, 5xx for upstream.
- Log server-side, stay vague to clients. Never leak stack traces or keys.
- Thin router, fat service. Test the logic without spinning up HTTP.

Your API is the contract the frontend trusts — version it and keep it stable.

Recap & what's next

You can now

- Say why the LLM belongs behind a backend
- Pick the right verb & status code
- Model requests/responses with Pydantic
- Expose a `POST /api/chat` with errors & a rate limit

Read the lecture notes, then start HW2 — build a `/api/chat` endpoint for your chat logic.

Next lecture

Lecture 4

Frontend, Reactivity & SSE — call this endpoint from the browser and stream the reply token by token.

Homework 2 — Backend & a streaming chat UI

L3 + L4

Start the backend now; you'll add the streaming browser front end in [Lecture 4](#).

This week (L3)

- A `POST /api/chat` wrapping your chat logic
- `ChatRequest` / `ChatResponse` Pydantic schemas
- Logic in a service, HTTP in the router
- Honest status codes: `422`, `429`, `5xx`

Design questions to answer first

- What's in the request body? The response?
- Which failures map to which status code?
- What does the API key never touch?

The endpoint you design here is the one the browser will call — and stream from — next lecture.

